

UNIVERSITY OF MALTA
FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF ARTIFICIAL INTELLIGENCE
ARI5118 – Deep Learning for Computer Vision



AI Usage Journal for ARI5118 – Deep Learning for Computer Vision

By

Favour Onyedikachi Umeh

Introduction

This journal provides a transparent, structured account of how I integrated artificial intelligence tools across all deliverables for the ARI5118 Deep Learning for Computer Vision assignment on Feature Pyramid Networks (FPN) and Multi-Scale Detection. The deliverables comprised: Study Notes on FPN architecture, a conceptual Quiz, a Streamlit-based interactive Principle Simulator, and a Code Walkthrough document. AI tools were not used uniformly across these deliverables—each tool was selected for a specific task based on what it demonstrably did well, and several tasks were deliberately kept human-authored. The following table summarises each tool and its primary role in the project.

I primarily used ChatGPT, Claude, and NotebookLM to understand the conceptual foundations of Feature Pyramid Networks. FPNs involve several interacting ideas — multi-scale feature extraction, top-down pathways, lateral connections, receptive fields, and feature fusion — and I found that using multiple models helped expose differences in explanation quality and technical depth. NotebookLM was particularly useful when working directly with uploaded research papers because it could summarize sections and connect ideas across documents.

For academic paper discovery, I used Elicit AI and ResearchRabbit. These tools helped identify related literature on FPN variants such as GraphFPN, LFPN, and DM-FPN, which became central to the literature review section of the assignment.

I used DeepSeek primarily for simulator code generation and experimentation. The simulator required visualizing how information propagates across pyramid levels and how feature fusion occurs. DeepSeek produced usable boilerplate code more quickly than other models for this task. The simulator interface itself was developed using Streamlit because it provided a lightweight way to deploy an interactive educational tool without needing full front-end development expertise.

Finally, most implementation and experimentation occurred within Google Colab. Beyond serving as the coding environment, Colab's integrated AI-assisted debugging and error explanation features were frequently used to interpret runtime errors, dependency conflicts, tensor shape mismatches, and visualization issues. The rationale for multi-tool use was deliberate: no single tool performed all tasks well.

Ethical Considerations

In developing this project, concerns about the trustworthiness of the AI's output, both in terms of factual accuracy and the existence of cited references, were the dominant challenges. Generative AI systems can be prone to presenting misleading and inaccurate technical details or providing non-existent citations with high confidence. In this context, because the work dealt with academic literature and technical deep learning concepts, all AI outputs were treated with suspicion until independently verified.

The largest ethical risk encountered was the hallucination of citations. While performing the literature search portion of the project, some of the AI-generated summaries cited papers that did not exist, incorrectly quoted author names, or fabricated content from existing papers. To combat this risk, every reference listed in the final assignment bibliography has been manually checked against the true paper and cross-referenced through official sources like IEEE Xplore, arXiv, Google Scholar, and publisher web pages.

Technical accuracy was a secondary consideration in this project. When using the AI to explain convolutional operations, receptive fields and feature fusion, certain key details about their implementation in a practical system were sometimes overlooked or presented imprecisely. In addition, the dimensions of a tensor are sometimes incorrectly identified, or feature map operations are generalized incorrectly, and it has been necessary to use my own reasoning to debug these cases.

Another ethical consideration was privacy. Because it's possible for generative AI systems to log user prompts and any uploaded files, the full draft of the assignment, custom source code and larger files from the project were never directly uploaded. Smaller examples or relevant snippets of code were provided instead when I was seeking debugging assistance.

Regarding academic integrity, it was important to intentionally avoid directly using the AI's output for the final critical analysis/interpretation parts of the project. Although the AI's explanations were utilized for idea generation and to better understand specific parts of the course material, all the literature synthesis, number breakdown, application details, and conclusions were written or restructured entirely by hand in a way that best reflected my own knowledge.

Methodology

My overall AI integration strategy followed an iterative workflow best described as Prompt → Verify → Modify → Rebuild. AI outputs were rarely used directly without substantial editing, testing, or reinterpretation.

- For the study notes, AI-assisted literature discovery was one of the most useful aspects of the workflow. Elicit AI and Research-Rabbit helped identify foundational and extension papers related to FPN architectures. These tools accelerated the discovery process significantly by surfacing citation relationships and visually mapping related research areas. Also, AI tools such as NotebookLM, ChatGPT and Claude were

primarily used to understand difficult conceptual topics related to FPN architecture. These included multi-scale feature extraction, top-down feature propagation, lateral connections, and pyramid level semantics. The study notes acted as the intellectual spine of the project, and it was important to develop a solid structure.

- The simulator was the most AI-intensive deliverable. It was also where AI caused the most friction. DeepSeek Coder generated the initial Streamlit scaffold, including the slider widgets, the feature map visualisation function, and the basic top-down propagation loop.
- For quiz creation, Notion AI was useful for generating question ideas and distractors. However, I avoided allowing AI to define the final question set independently because some generated questions oversimplified technical concepts or included incorrect distractors. Quiz creation was the deliverable with the least AI involvement. The main contribution was distractor generation, which saved approximately 30 minutes of effort.
- For the code walkthrough, AI tools such as DeepSeek were mainly used to explain library functions, clarify tensor operations, and reformat explanations into more readable sequences. I also used DeepSeek to generate inline comments for dense code blocks, check that my explanatory prose accurately described what each function did; suggesting a clearer order for presenting the code sections.

Prompts and Responses

1. Claude

"You are an academic writer specializing in computer vision coursework, tasked with developing a comprehensive assignment on Feature Pyramid Networks (FPN) and Multi-Scale Detection that meets A-grade standards.

Your assignment follows this table of contents structure:

- 1. Introduction*
- 2. Literature Review*
- 3. Technical Foundations and Core Algorithms*
- 4. Real-World Applications in Computer Vision*
- 5. Reference: Sources & Further Reading*

Your core objective: Create a detailed yet concise assignment that demonstrates both technical depth and clear communication of complex concepts.

Section Requirements:

Introduction

Establish the context and significance of multi-scale detection in computer vision. Clearly define the "scale problem"—why objects of different sizes are challenging to detect reliably. Position FPNs as a solution. End with a thesis statement that outlines what the assignment will cover.

Literature Review

Synthesize 4-6 key academic sources on FPNs and multi-scale detection. For each source, explain its core contribution and relevance to your topic. Reference the project brief (ARI5118 25-26 - Project Brief.docx) to ensure you're addressing verified, relevant papers. Organize chronologically or thematically to show how the field evolved.

Technical Foundations and Core Algorithms

This is your deepest section. Cover:

- The architectural design of FPNs: top-down pathway, lateral connections, and scale-specific detection heads

- *Mathematical foundations underlying multi-scale feature extraction*
- *A step-by-step numerical example showing how FPN processes features at different pyramid levels*
- *How FPNs differ from alternative multi-scale approaches*
- *Common misconceptions and why they're incorrect*

Use clear analogies alongside formal explanations to bridge intuition and rigor.

Real-World Applications in Computer Vision:

Provide concrete, specific applications where FPNs and multi-scale detection excel. Include examples like object detection in aerial imagery, small object detection, and scenarios where scale variation is critical. Explain how the technical concepts from the previous section directly enable these applications.

Reference: Sources & Further Reading:

Compile a properly formatted bibliography. Include verified academic papers, with full citations (author, year, title, venue) and URLs where applicable.

Tone & Style:

- *Write for an academic audience with computer vision background*
- *Balance technical precision with accessibility—avoid unnecessary jargon, but use domain terminology correctly*
- *Use figures, diagrams, or mathematical notation where they clarify concepts*
- *Keep explanations concise; avoid redundancy between sections*

Quality Standards:

- *Each section should be substantive but focused—no filler*
- *Ensure internal consistency: concepts introduced in Technical Foundations should be applied in Applications*
- *Demonstrate critical thinking: don't just describe FPNs, explain why they matter and what problems they solve*
- *Verify all claims with cited sources*

Develop this assignment to submission-ready quality in a single, cohesive document.”

Result:

Here's an overview of what each section delivers:

Section 1 — Introduction opens with the practical stakes of multi-scale detection (surveillance, satellite imagery, medical imaging), formally defines the scale problem as the tension between spatial resolution and semantic depth across backbone layers, critiques pre-FPN approaches, and positions FPN and its successors within a clear thesis arc.

Section 2 — Literature Review provides critical synthesis of all five papers you uploaded, organized thematically. It traces the field from Lin et al.'s foundational architecture through GraphFPN's challenge to fixed topology, LFPN's latent-space long-range dependency modelling, MSRA-FPN's attack on information decay, and DM-FPN's domain adaptation for remote sensing. Each entry explains the paper's core contribution and its specific relevance to the assignment's arguments.

Section 3 — Technical Foundations is the deepest section. It covers the bottom-up/top-down/lateral architecture with precise mathematical notation, the pyramid level assignment formula with derivation, and a fully worked numerical example tracing feature propagation from C3/C4 through upsampling, lateral projection, element-wise merge, and the final 3×3 smoothing — including a concrete 2×2 numerical illustration. Five common misconceptions are each diagnosed and corrected with technical justification.

Section 4 — Applications grounds the technical material in geospatial detection (DOTA benchmark), small-object surveillance, instance segmentation in pathology, and autonomous driving perception — each connecting explicitly back to the architectural mechanisms in Section 3.

Section 5 — References provides full citations with DOIs and URLs for all five primary papers, plus ten additional references, formatted for academic submission.

2. Elicit AI: “Give me papers related to the topic: Feature Pyramid Networks in Computer Vision.”

Result:

The screenshot shows the Elicit AI interface. At the top, there's a search bar with the query 'Feature Pyramid Networks and Multi-Scale Detection'. Below the search bar, there's a table of results. The table has columns for 'Paper', '10 sources', and 'Summary'. The first row is 'Feature Pyramid Networks for Object Detection' by Tsung-Yi Lin, Piotr Dollár, Ross B. Girshick, Kaiming He, Bharath Hariharan, and 1 more. The second row is 'GraphFPN: Graph Feature Pyramid Network for Object Detection' by Gangming Zhao, Weifeng Ge, Yizhou Yu. The third row is 'Multi-Scale Residual Aggregation Feature Pyramid Network for Object Detection' by Hongyang Wang, Tiejun Wang. To the right of the table, there's a 'Chat with papers' section with an 'Answer' box. The answer box contains a summary of the papers, stating that Feature Pyramid Networks (FPNs) are a foundational architecture that effectively addresses multi-scale object detection by exploiting the inherent hierarchical structure of deep convolutional networks. It also mentions that the evidence is substantial and consistent, and that the original FPN approach achieved state-of-the-art single-model results on the COCO detection benchmark, running at 5 FPS on GPU.

3. Deepseek

“You are a Python/Streamlit developer building an interactive simulator for Feature Pyramid Networks (FPNs) designed to run on Snowflake with CPU-only execution.

Your task: Create a fully functional, production-ready Streamlit application that allows users to interactively explore FPN multi-scale object detection through real-time parameter adjustment and side-by-side comparison visualization.

Core Requirements:

1. Interactive Parameter Controls: Build sliders and input fields for users to adjust:

- Anchor scales (e.g., 32^2 to 512^2 pixels)
- Aspect ratios (1:2, 1:1, 2:1)
- Convolutional kernel sizes (1x1, 3x3)
- Channel dimensions (e.g., 256)
- Confidence thresholds for detection output
- Any other FPN-relevant hyperparameters from your study notes

2. Real-Time Visual Feedback: When parameters change, the visualization must update immediately to show the impact on feature pyramid construction, anchor generation, and detection output.

3. *Comparison Mode: Implement a side-by-side view that allows users to:*

- *Compare two different parameter configurations simultaneously*
- *Display results for different FPN variants or architectural approaches (reference Table 3.2 from your study notes comparing FPN with SSD/MS-CNN, PANet, GraphFPN, LFPN, featurized image pyramids)*

- *Show detection differences, computation metrics, or pyramid structure variations*

4. *CPU-Only Execution: The simulator must run entirely on CPU. No GPU dependencies allowed. Optimize for inference speed and memory efficiency.*

5. *Code Quality:*

- *Write clean, well-commented code*
- *Include a `requirements.txt` with all dependencies pinned*
- *Ensure peers can run the application with zero additional setup beyond installing requirements*
- *Make the code executable and test it end-to-end*

6. *Snowflake Integration: If your workflow requires Snowflake connectivity (e.g., loading sample data or logging results), include minimal, optional integration—but the core simulator must function standalone.*

Input Materials:

- *Reference your FPN study notes (`FPN\_Study\_Notes\_ARI5118.pdf`) for technical accuracy on architecture, parameters, and comparative context*
- *Use sample images or synthetic data for demonstration (no large external downloads)*

Output:

A complete, runnable Streamlit application with supporting files (requirements.txt, any data files needed). The app should be immediately deployable and usable by your peers for exploring FPN behavior across different configurations.

Result:

```
import streamlit as st
import numpy as np
from PIL import Image, ImageDraw, ImageFont
import math
from typing import List, Tuple, Dict, Any

#
=====
# Synthetic Image and Ground Truth Generation
#
=====

@st.cache_data
def get_synthetic_image_and_gt(size: int = 256) -> Tuple[np.ndarray, List[Dict]]:
    """
    Generate a synthetic image with objects of varying sizes and aspect ratios.
    Returns:
```

```

        image: RGB numpy array of shape (H, W, 3)
        ground_truths: list of dicts with keys 'bbox' (x1,y1,x2,y2), 'label', 'area'
    """
    img = Image.new('RGB', (size, size), color=(240, 240, 240))
    draw = ImageDraw.Draw(img)

    # Ground truth objects: small, medium, large
    objects = [
        {'label': 'small_circle', 'bbox': (40, 40, 70, 70), 'color': (200, 50, 50)},
        {'label': 'medium_rect', 'bbox': (100, 80, 180, 140), 'color': (50, 150, 50)},
        {'label': 'large_rect', 'bbox': (180, 170, 250, 240), 'color': (50, 50, 200)},
    ]

    for obj in objects:
        draw.rectangle(obj['bbox'], outline=obj['color'], width=2)
        # fill slightly to help detection
        draw.rectangle(obj['bbox'], fill=(*obj['color'], 50))

    # optional: add small clutter
    draw.rectangle((200, 50, 220, 65), outline=(100, 100, 100), width=1)

    img_np = np.array(img)
    ground_truths = []
    for obj in objects:
        x1, y1, x2, y2 = obj['bbox']
        area = (x2 - x1) * (y2 - y1)
        ground_truths.append({
            'bbox': np.array([x1, y1, x2, y2], dtype=np.float32),
            'label': obj['label'],
            'area': area
        })
    return img_np, ground_truths

#
=====
=====
# Anchor Generation (FPN style per pyramid level)
#
=====
=====

def generate_anchors_for_level(
    feat_h: int, feat_w: int, stride: int,
    base_scale: float, aspect_ratios: List[float],
    img_h: int, img_w: int
) -> np.ndarray:
    """
    Generate anchors for a single pyramid level.
    Returns: (N, 4) array of (x1, y1, x2, y2) in image coordinates.
    """
    centers_x = (np.arange(feat_w) + 0.5) * stride
    centers_y = (np.arange(feat_h) + 0.5) * stride
    xx, yy = np.meshgrid(centers_x, centers_y)
    centers = np.stack([xx.ravel(), yy.ravel()], axis=1) # (num_cells, 2)

```

```

anchors = []
for ar in aspect_ratios:
    w = base_scale * np.sqrt(ar)
    h = base_scale / np.sqrt(ar)
    half_w, half_h = w / 2.0, h / 2.0
    x1 = centers[:, 0] - half_w
    y1 = centers[:, 1] - half_h
    x2 = centers[:, 0] + half_w
    y2 = centers[:, 1] + half_h
    boxes = np.stack([x1, y1, x2, y2], axis=1)
    anchors.append(boxes)

anchors = np.vstack(anchors) # (num_cells * len(aspect_ratios), 4)
# Clip to image boundaries
anchors[:, 0] = np.clip(anchors[:, 0], 0, img_w - 1)
anchors[:, 1] = np.clip(anchors[:, 1], 0, img_h - 1)
anchors[:, 2] = np.clip(anchors[:, 2], 0, img_w - 1)
anchors[:, 3] = np.clip(anchors[:, 3], 0, img_h - 1)
# Filter degenerate boxes where x1>=x2 or y1>=y2
valid = (anchors[:, 2] > anchors[:, 0]) & (anchors[:, 3] > anchors[:, 1])
return anchors[valid]

def compute_all_anchors(
    img_h: int, img_w: int,
    anchor_scale_factor: float,
    aspect_ratios: List[float]
) -> Tuple[List[np.ndarray], List[Dict]]:
    """
    Generate anchors for FPN levels P2-P5 (stride 4,8,16,32).
    Returns:
        anchors_per_level: list of (N_i, 4) arrays
        level_metadata: list of dicts with level info
    """
    pyramid_configs = [
        {'level': 'P2', 'stride': 4, 'base_scale': 32},
        {'level': 'P3', 'stride': 8, 'base_scale': 64},
        {'level': 'P4', 'stride': 16, 'base_scale': 128},
        {'level': 'P5', 'stride': 32, 'base_scale': 256},
    ]
    anchors_per_level = []
    level_metadata = []

    for cfg in pyramid_configs:
        stride = cfg['stride']
        feat_h = img_h // stride
        feat_w = img_w // stride
        base_scale = cfg['base_scale'] * anchor_scale_factor
        anchors = generate_anchors_for_level(
            feat_h, feat_w, stride, base_scale, aspect_ratios,
            img_h, img_w
        )
        anchors_per_level.append(anchors)
        level_metadata.append({
            'name': cfg['level'],
            'stride': stride,

```

```

        'feat_size': f'{feat_h} × {feat_w}',
        'num_anchors': len(anchors),
        'base_scale': base_scale
    })
    return anchors_per_level, level_metadata

#
=====
# IoU and Detection Simulation
#
=====

def compute_iou_matrix(boxes: np.ndarray, gt_boxes: np.ndarray) -> np.ndarray:
    """boxes: (N,4), gt_boxes: (M,4) -> IoU matrix (N, M)"""
    N = boxes.shape[0]
    M = gt_boxes.shape[0]
    ious = np.zeros((N, M), dtype=np.float32)
    for i in range(N):
        x1 = max(boxes[i, 0], gt_boxes[:, 0])
        y1 = max(boxes[i, 1], gt_boxes[:, 1])
        x2 = min(boxes[i, 2], gt_boxes[:, 2])
        y2 = min(boxes[i, 3], gt_boxes[:, 3])
        inter = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)
        area1 = (boxes[i, 2] - boxes[i, 0]) * (boxes[i, 3] - boxes[i, 1])
        area2 = (gt_boxes[:, 2] - gt_boxes[:, 0]) * (gt_boxes[:, 3] - gt_boxes[:, 1])
        union = area1 + area2 - inter
        ious[i, :] = inter / (union + 1e-6)
    return ious

def non_max_suppression(boxes: np.ndarray, scores: np.ndarray, iou_thresh: float = 0.5) -> np.ndarray:
    """Standard greedy NMS."""
    if len(boxes) == 0:
        return np.array([])
    indices = np.argsort(scores)[-1:]
    keep = []
    while len(indices) > 0:
        current = indices[0]
        keep.append(current)
        if len(indices) == 1:
            break
        rest = indices[1:]
        ious = np.array([
            compute_iou_matrix(boxes[current:current+1], boxes[r:r+1])[0,0]
            for r in rest
        ])
        indices = rest[ious <= iou_thresh]
    return np.array(keep)

def simulate_detections(
    anchors_per_level: List[np.ndarray],
    ground_truths: List[Dict],

```

```

confidence_thresh: float,
variant: str,
channel_dim: int,
kernel_size: int,
lateral_connections: bool
) -> Tuple[np.ndarray, np.ndarray, Dict]:
"""
Simulate detection scores based on anchor-GT IoU and variant-specific effects.
Returns:
    final_boxes (K,4): detected boxes after NMS
    final_scores (K,): confidence scores
    stats: dict of metrics
"""
all_anchors = np.vstack(anchors_per_level)
gt_boxes = np.array([gt['bbox'] for gt in ground_truths])
if len(gt_boxes) == 0:
    return np.empty((0,4)), np.empty(0), {'total_anchors': len(all_anchors), 'num_detections': 0, 'mean_iou':
0.0, 'approx_mAP': 0.0}
# IoU with ground truth
iou_matrix = compute_iou_matrix(all_anchors, gt_boxes)
max_iou_per_anchor = iou_matrix.max(axis=1)
best_gt_idx = iou_matrix.argmax(axis=1)

# --- Base confidence from IoU ---
base_conf = max_iou_per_anchor.copy()

# --- Variant-specific adjustments (based on Table 3.2 from study notes) ---
# Determine small object indices (area < 32*32 = 1024)
small_gt_indices = [i for i, gt in enumerate(ground_truths) if gt['area'] < 1024]
is_small_anchor = np.isin(best_gt_idx, small_gt_indices)

if variant == "FPN (Lin et al. 2017)":
    # Standard FPN: strong performance across scales
    conf = base_conf * (0.9 + 0.2 * (channel_dim / 256.0))
    if kernel_size == 3:
        conf *= 1.05 # 3x3 smoothing helps
    if not lateral_connections:
        conf *= 0.7 # severe degradation without lateral connections
elif variant == "SSD / MS-CNN":
    # No top-down fusion, shallow layers are semantically weak
    conf = base_conf * 0.75
    conf[is_small_anchor] *= 0.6 # small objects suffer heavily
    if not lateral_connections:
        conf *= 0.9 # already weak, less extra impact
elif variant == "PANet (Liu et al. 2018)":
    # Extra bottom-up path boosts small/medium object recall
    conf = base_conf * (1.0 + 0.15 * (channel_dim / 256.0))
    conf[is_small_anchor] *= 1.2
    if kernel_size == 3:
        conf *= 1.03
elif variant == "GraphFPN (Zhao et al. 2021)":
    # Graph-based interaction improves small objects
    conf = base_conf * 1.05
    conf[is_small_anchor] *= 1.25
    if not lateral_connections:
        conf *= 0.85

```

```

elif variant == "LFPN (Xie et al. 2022)":
    # Latent space modelling captures long-range dependencies
    conf = base_conf * 1.08
    # extra boost for medium and large
    conf[~is_small_anchor] *= 1.1
else:
    conf = base_conf

# Clip confidence to [0,1]
conf = np.clip(conf, 0.0, 1.0)

# Apply confidence threshold
keep = conf >= confidence_thresh
if not np.any(keep):
    return np.empty((0,4)), np.empty(0), {'total_anchors': len(all_anchors), 'num_detections': 0, 'mean_iou':
0.0, 'approx_mAP': 0.0}

boxes = all_anchors[keep]
scores = conf[keep]

# Non-maximum suppression
nms_keep = non_max_suppression(boxes, scores, iou_thresh=0.5)
final_boxes = boxes[nms_keep]
final_scores = scores[nms_keep]

# Compute approximate mAP: mean IoU of matched detections to ground truth
if len(final_boxes) > 0:
    final_iou_matrix = compute_iou_matrix(final_boxes, gt_boxes)
    best_match_iou = final_iou_matrix.max(axis=1).mean()
else:
    best_match_iou = 0.0

stats = {
    'total_anchors': len(all_anchors),
    'num_detections': len(final_boxes),
    'mean_iou': best_match_iou,
    'approx_mAP': best_match_iou * 0.9 # rough lower bound mAP proxy
}
return final_boxes, final_scores, stats

#
=====
=====
# Visualization Helpers
#
=====
=====

def draw_boxes_on_image(image_np: np.ndarray, boxes: np.ndarray, scores: np.ndarray,
                        color=(255, 0, 0), label_prefix: str = "obj") -> np.ndarray:
    """Draw bounding boxes with scores on image."""
    img = Image.fromarray(image_np).convert('RGB')
    draw = ImageDraw.Draw(img)
    try:
        font = ImageFont.load_default()

```

```

except:
    font = None

for i, box in enumerate(boxes):
    x1, y1, x2, y2 = box.astype(int)
    draw.rectangle([x1, y1, x2, y2], outline=color, width=2)
    score_text = f"{label_prefix}: {scores[i]:.2f}" if i < len(scores) else label_prefix
    draw.text((x1, max(y1-12, 0)), score_text, fill=color, font=font)
return np.array(img)

def render_pyramid_summary(level_metadata: List[Dict]) -> str:
    """Format FPN pyramid level summary as markdown."""
    lines = ["| Level | Stride | Feat. Size | Anchors | Base Scale |", "|-----|-----|-----|-----|-----|"]
    for m in level_metadata:
        lines.append(f"| {m['name']} | {m['stride']} | {m['feat_size']} | {m['num_anchors']} | {m['base_scale']:.1f} |")
    return "\n".join(lines)

#
=====
# Streamlit App: Side-by-Side Comparison
#
=====

st.set_page_config(page_title="FPN Simulator", layout="wide")
st.title("🔍 Feature Pyramid Network (FPN) Simulator")
st.markdown("Interactive exploration of multi-scale object detection | *CPU-only, no training required*")

# Load synthetic image and ground truth (cached)
img_np, ground_truths = get_synthetic_image_and_gt(size=256)
orig_img = Image.fromarray(img_np)

# Helper to compute and display a single configuration
def show_configuration(col, config_name: str, default_params: Dict[str, Any]):
    with col:
        st.subheader(f"⚙️ {config_name}")

        # --- Parameter inputs in an expander ---
        with st.expander("FPN Parameters", expanded=True):
            anchor_scale = st.slider("Anchor Scale Factor", 0.5, 2.0, default_params.get('anchor_scale', 1.0), 0.05,
key=f"{config_name}_anchor_scale")
            aspect_ratios = st.multiselect(
                "Aspect Ratios (width/height)",
                options=[0.5, 1.0, 2.0],
                default=default_params.get('aspect_ratios', [0.5, 1.0, 2.0]),
                format_func=lambda x: f"{x}:1" if x>=1 else f"1:{int(1/x)}",
                key=f"{config_name}_aspect"
            )
            if not aspect_ratios:
                aspect_ratios = [1.0]

```

```

        channel_dim = st.selectbox("Channel Dimension (Latent fusion dim)", [128, 256, 512], index=1,
key=f"{config_name}_channels")
        kernel_size = st.selectbox("Lateral / Smoothing Kernel", [1, 3], format_func=lambda x: f"{x}{x}{x}",
key=f"{config_name}_kernel")
        confidence_thresh = st.slider("Confidence Threshold", 0.0, 1.0, default_params.get('confidence', 0.5),
0.02, key=f"{config_name}_conf")
        lateral_connections = st.checkbox("Lateral Connections (FPN essential)",
value=default_params.get('lateral', True), key=f"{config_name}_lateral")
        variant = st.selectbox(
            "FPN Variant / Model",
            ["FPN (Lin et al. 2017)", "SSD / MS-CNN", "PANet (Liu et al. 2018)", "GraphFPN (Zhao et al.
2021)", "LFPN (Xie et al. 2022)"],
            index=0,
            key=f"{config_name}_variant"
        )

# --- Compute anchors and detections ---
anchors_per_level, level_metadata = compute_all_anchors(
    img_h=256, img_w=256,
    anchor_scale_factor=anchor_scale,
    aspect_ratios=aspect_ratios
)

final_boxes, final_scores, stats = simulate_detections(
    anchors_per_level, ground_truths,
    confidence_thresh=confidence_thresh,
    variant=variant,
    channel_dim=channel_dim,
    kernel_size=kernel_size,
    lateral_connections=lateral_connections
)

# --- Visualization: Ground Truth + Detections ---
gt_img = draw_boxes_on_image(img_np, [gt['bbox'] for gt in ground_truths], [1.0]*len(ground_truths),
color=(0,255,0), label_prefix="GT")
if len(final_boxes) > 0:
    result_img = draw_boxes_on_image(gt_img, final_boxes, final_scores, color=(255,50,50),
label_prefix="det")
else:
    result_img = gt_img

col.image(result_img, caption=f"Detections (n={stats['num_detections']})", use_container_width=True)

# --- Metrics and Pyramid Summary ---
st.metric("Total Anchors", f"{stats['total_anchors']:,}")
col1, col2 = st.columns(2)
col1.metric("Detections", stats['num_detections'])
col2.metric("Approx. mAP (proxy)", f"{stats['approx_mAP']:.2f}")
st.caption(f"Mean IoU with GT: {stats['mean_iou']:.3f}")

with st.expander("🔍 Feature Pyramid Details (P2-P5)":
    st.markdown(render_pyramid_summary(level_metadata))
    st.caption("Each level's anchors are scale-specific. Lateral connections + top-down fusion enrich
high-resolution features.")

# Cost estimation (floating point ops proxy)

```

```

total_conv_ops = sum( (256//((2**i))**2 * channel_dim * (kernel_size**2) for i in range(2,6) ) // 1000
st.caption(f"  Approx. conv FLOPs ( $\times 10^3$ ): {total_conv_ops} | Lateral active: {lateral_connections} |
Kernel: {kernel_size}x{kernel_size}")

return {
    'variant': variant,
    'detections': len(final_boxes),
    'mAP': stats['approx_mAP'],
    'anchors': stats['total_anchors']
}

# --- Side-by-Side Layout ---
st.markdown("---")
st.subheader("  Side-by-Side Configuration Comparison")
st.markdown("Adjust parameters on each side independently to compare FPN variants, anchor scales, and
fusion strategies.")

left_col, right_col = st.columns(2, gap="large")
with left_col:
    left_stats = show_configuration(left_col, "Configuration A",
                                   default_params={'anchor_scale': 1.0, 'confidence': 0.5, 'lateral': True, 'aspect_ratios':
[0.5, 1.0, 2.0]})
with right_col:
    right_stats = show_configuration(right_col, "Configuration B",
                                    default_params={'anchor_scale': 1.2, 'confidence': 0.45, 'lateral': False, 'aspect_ratios':
[1.0, 2.0]})

# --- Comparison Summary ---
st.markdown("---")
st.subheader("  Comparison Summary (according to Table 3.2 from FPN Literature)")
comp_col1, comp_col2, comp_col3 = st.columns(3)
comp_col1.metric("Variant A", left_stats['variant'])
comp_col1.metric("Variant B", right_stats['variant'])
comp_col2.metric("Detections A", left_stats['detections'])
comp_col2.metric("Detections B", right_stats['detections'])
comp_col3.metric("Approx. mAP A", f"{left_stats['mAP']:.2f}")
comp_col3.metric("Approx. mAP B", f"{right_stats['mAP']:.2f}")

st.info("  **Insights from study notes** – FPN’s lateral connections + top-down pathway enable strong
semantics at all scales (Lin et al. 2017). "
"GraphFPN uses superpixel-adaptive graphs to boost small objects; SSD lacks top-down fusion, harming
small-object recall. "
"Compare side-by-side to observe how anchor scale, lateral connections, and variant choice affect
detections and anchor density.")

st.caption("CPU-only simulation | Anchors computed in real-time | Detection confidence simulated from
anchor-GT IoU with variant-specific gains (aligned with COCO-style behaviour)")

```

Improvements, Errors and Contributions

AI tools improved the project in several meaningful ways throughout the development process. One of the biggest advantages was how much faster they helped me understand difficult concepts. FPN architectures involve complex ideas such as CNN hierarchies, multi-scale feature representations, semantic information flow, and feature fusion. Instead of relying only on research papers, AI explanations gave me different perspectives and simpler breakdowns, which helped me identify gaps in my understanding much more quickly.

Tools like Elicit and ResearchRabbit also saved a significant amount of time during the literature review phase. They made it easier to discover relevant papers, trace citation relationships, and identify newer FPN variants that I might not have found as efficiently through manual searches alone.

AI-generated boilerplate code was useful during the simulator development stage as well. It helped me quickly build the initial Streamlit structure, create visualization layouts, suggest tensor operation logic, and better understand how certain library functions worked.

In addition, Google Colab's built-in AI "explain and fix-error" feature, together with debugging support from ChatGPT, helped me troubleshoot several technical issues, including tensor dimension mismatches, interpolation errors, rendering problems in Streamlit, and inconsistencies in feature fusion outputs.

AI tools were also helpful for smaller but important tasks, such as generating realistic distractors for multiple-choice questions and improving awkward or unclear wording in technical explanations, making the final presentation of the project more polished and easier to understand.

Individual Reflection

Looking back on the project, AI tools were genuinely helpful in several areas, particularly during literature discovery, conceptual clarification, and the early stages of simulator development. Tools such as Elicit and ResearchRabbit reduced the amount of manual effort needed to find relevant papers, explore citation relationships, and identify newer FPN variants. This made the research process feel more manageable and less time-consuming.

Similarly, tools like ChatGPT and Claude helped me understand difficult concepts more quickly by explaining ideas in simpler language and presenting multiple perspectives on the same topic. For complex areas such as feature fusion, multi-scale representations, and CNN hierarchies, these explanations often helped me bridge gaps in understanding faster than reading technical papers alone.

However, one of the most important lessons from this project was realizing that AI assistance still requires constant verification. I spent a surprising amount of time correcting mistakes in AI-generated explanations and code. Some technical responses sounded extremely convincing but contained subtle inaccuracies, particularly around tensor dimensions, interpolation

behaviour, and feature fusion operations. In several cases, debugging AI-generated simulator code took longer than expected, and there were moments when rewriting sections manually was actually more efficient than fixing the generated output.

What stood out to me most was how confidently AI systems could present incorrect information. I noticed that AI tools performed best when handling focused, clearly defined tasks with outputs that could be easily verified, such as arithmetic checks, boilerplate code generation, distractor creation, or minor debugging tasks. They were much less reliable for maintaining deep conceptual accuracy across long conversations.

I experienced this particularly with Claude. Early explanations about FPN architectures were accurate and useful, but as the discussion became longer and more specific, inconsistencies and inaccuracies gradually appeared. I cannot say for certain whether this was caused by context limitations or the model adapting too heavily to my phrasing, but it reinforced an important lesson: AI works better as a tool for short, focused consultations rather than as a consistently reliable long-term collaborator.

Overall, the time savings from AI were real, but unevenly distributed across the project. Literature discovery, distractor generation, phrasing improvements, and smaller coding tasks were noticeably accelerated. On the other hand, the simulator core implementation and mathematical worked examples required substantial independent effort, and AI support in these areas sometimes introduced additional debugging and verification work.

In practical terms, I would estimate that AI saved approximately three to four hours of work across the project, while adding one to two hours of verification, debugging, and correction. The overall impact was still positive, but the benefit was smaller and more nuanced than I initially expected. Most importantly, this experience taught me that AI should be treated as a supportive tool rather than an authoritative source, especially in technically demanding academic work.

References

- AI tools Used:

Tool	Primary Role in Project
Claude	Conceptual clarification, mathematical verification, prose review
ChatGPT and Notion AI	Quiz distractor generation, cross-checking architectural claims
NotebookLM	Annotated synthesis of Lin et al. (2017) and Wang & Wang (2023)

Elicit AI and Research Rabbit	Structured academic literature search for FPN extension papers
DeepSeek Coder	Streamlit simulator scaffolding and initial code generation
Google Colab AI features	Inline error explanation and debugging suggestions for the simulator

- Documentations and Official Resources Sources:

1. IEEE Xplore
2. Google Scholar
3. arXiv
4. Official Streamlit Documentation
5. Official PyTorch Documentation

- Academic Papers:

1. T. -Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, "Feature Pyramid Networks for Object Detection," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017 <https://doi.org/10.1109/CVPR.2017.106>
2. Zhang, X., Zhu, K., Chen, G., Tan, X., Zhang, L., Dai, F., Liao, P., & Gong, Y. (2019). Geospatial Object Detection on High Resolution Remote Sensing Imagery Based on Double Multi-Scale Feature Pyramid Network. Remote Sensing, 11(7), 755. <https://doi.org/10.3390/rs11070755>
3. Wang, H., & Wang, T. (2023). Multi-Scale Residual Aggregation Feature Pyramid Network for Object Detection. Electronics, 12(1), 93. <https://doi.org/10.3390/electronics12010093>
4. Tang, L., Tang, W., Qu, X., Han, Y., Wang, W., & Zhao, B. (2022). A Scale-Aware Pyramid Network for Multi-Scale Object Detection in SAR Images. Remote Sensing, 14(4), 973. <https://doi.org/10.3390/rs14040973>
5. Golnaz Ghiasi, Tsung-Yi Lin, Ruoming Pang, & Quoc V. Le. (2019). NAS-FPN: Learning Scalable Feature Pyramid Architecture for Object Detection. CoRR, abs/1904.07392. <https://doi.org/10.48550/arxiv.1904.07392>
6. G. Zhao, W. Ge and Y. Yu, "GraphFPN: Graph Feature Pyramid Network for Object Detection," 2021 IEEE/CVF International Conference on Computer Vision (ICCV), Montreal, QC, Canada, 2021, pp. 2743-2752, doi: <https://doi.org/10.1109/ICCV48922.2021.00276>.
7. J. Xie, Y. Pang, J. Nie, J. Cao and J. Han, "Latent Feature Pyramid Network for Object Detection," in IEEE Transactions on Multimedia, vol. 25, pp. 2153-2163, 2023, doi: <https://doi.org/10.1109/TMM.2022.3143707>.